Real-World Embedded Use Cases: Smart Pointers

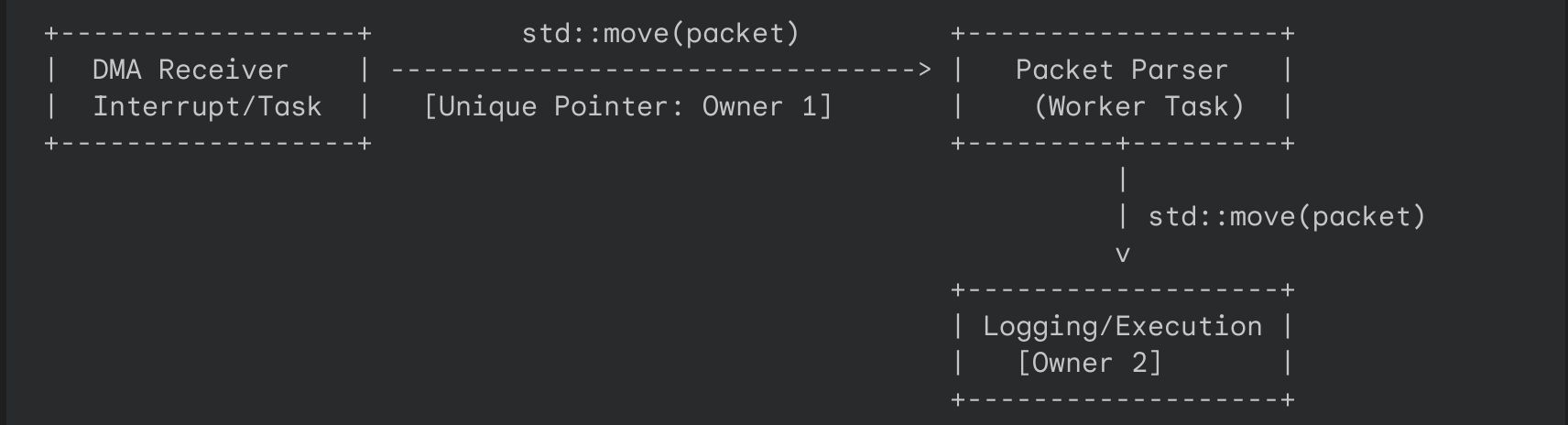
To truly master modern C++ in systems and IoT architectures, we must look beyond basic syntactical examples and see how these pointers solve concurrency, lifecycle, and safety challenges in real-world firmware.

Below are three highly practical, production-ready embedded software patterns showing how `std::unique_ptr`, `std::shared_ptr`, and `std::weak_ptr` prevent memory corruption and guarantee thread safety in multi-tasking environments.

1. `std::unique_ptr` Use Case: Zero-Copy DMA RX Packet Pipeline

Imagine an embedded gateway (like a smart home hub) receiving rapid bursts of telemetry data over a UART/SPI DMA channel. The raw bytes are loaded by hardware directly into memory. Once a full frame is received, it must be:

1. **Parsed:** Validated for CRC and structured into a packet object.
2. **Processed:** Acted upon by the local control loop.
3. **Logged:** Serialized to flash memory.



The Architectural Problem

- If we pass this data by **copying**, we waste precious CPU cycles and SRAM copying bytes between thread queues.
- If we pass **raw pointers** between threads, we run the massive risk of a race condition: what if the receiver task overwrites or deletes the packet buffer while the parser task is still reading it?
- We need a way to pass *exclusive ownership* of the buffer from one task to another.

The Modern C++ Solution

Use `std::unique_ptr<RxPacket>`. Because a `std::unique_ptr` **cannot be copied**, it can only be **moved** (`std::move`). When Task A moves the pointer to Task B, Task A's local pointer is instantly set to `nullptr` by the compiler. It is mathematically impossible for Task A to accidentally read/write to that buffer ever again. Ownership has been cleanly and safely transferred with zero memory copying.

Code Implementation

```
#include <iostream>
#include <memory>
#include <queue>
#include <thread>
#include <utility>
#include <vector>

struct RxPacket {
    uint16_t packetId;
    std::vector<uint8_t> payload;

    RxPacket(uint16_t id, std::vector<uint8_t> data)
        : packetId(id), payload(std::move(data)) {
        std::cout << "[ALLOC] Packet #" << packetId << " allocated in RAM.\n";
    }

    ~RxPacket() {
```

```
        std::cout << "[FREE] Packet #" << packetId << " safely deallocated.\n";
    }
};

// Simulated thread-safe queue holding exclusive ownership of packets
std::queue<std::unique_ptr<RxPacket>> transitQueue;

void dmaReceiverTask() {
    std::cout << "[DMA Task] Interrupt triggered! Frame complete.\n";

    // Allocate packet exclusively
    auto newPacket = std::make_unique<RxPacket>(101, std::vector<uint8_t>{0xAA, 0xBB, 0xCC})

    // We can write to it safely here...
    newPacket->payload.push_back(0xDD);

    std::cout << "[DMA Task] Passing packet to transit queue...\n";

    // Transfer exclusive ownership to the queue.
    // After this line, 'newPacket' is nullptr and this thread can no longer access it!
    transitQueue.push(std::move(newPacket));
}

void packetParserTask() {
    if (transitQueue.empty()) return;

    // Pop the exclusive ownership from the queue into our local variable
    std::unique_ptr<RxPacket> activePacket = std::move(transitQueue.front());
    transitQueue.pop();

    std::cout << "[Parser Task] Safely acquired exclusive lock on Packet #" << activePacket-
    std::cout << "  -> Packet payload size: " << activePacket->payload.size() << " bytes.\n"

    // When 'activePacket' goes out of scope here, it is automatically destroyed!
    std::cout << "[Parser Task] Work complete. Leaving scope...\n";
}

int main() {
    std::cout << "=== SYSTEM BOOT: DMA PIPELINE RUN ===\n\n";

    dmaReceiverTask();
    std::cout << "\n--- Queue holds ownership of packet ---\n\n";
    packetParserTask();

    std::cout << "\n=== DIAGNOSTIC RUN ENDED ===" << std::endl;
    return 0;
}
```

2. `std::shared_ptr` Use Case: Multi-Tasking Weather Station Data Broker

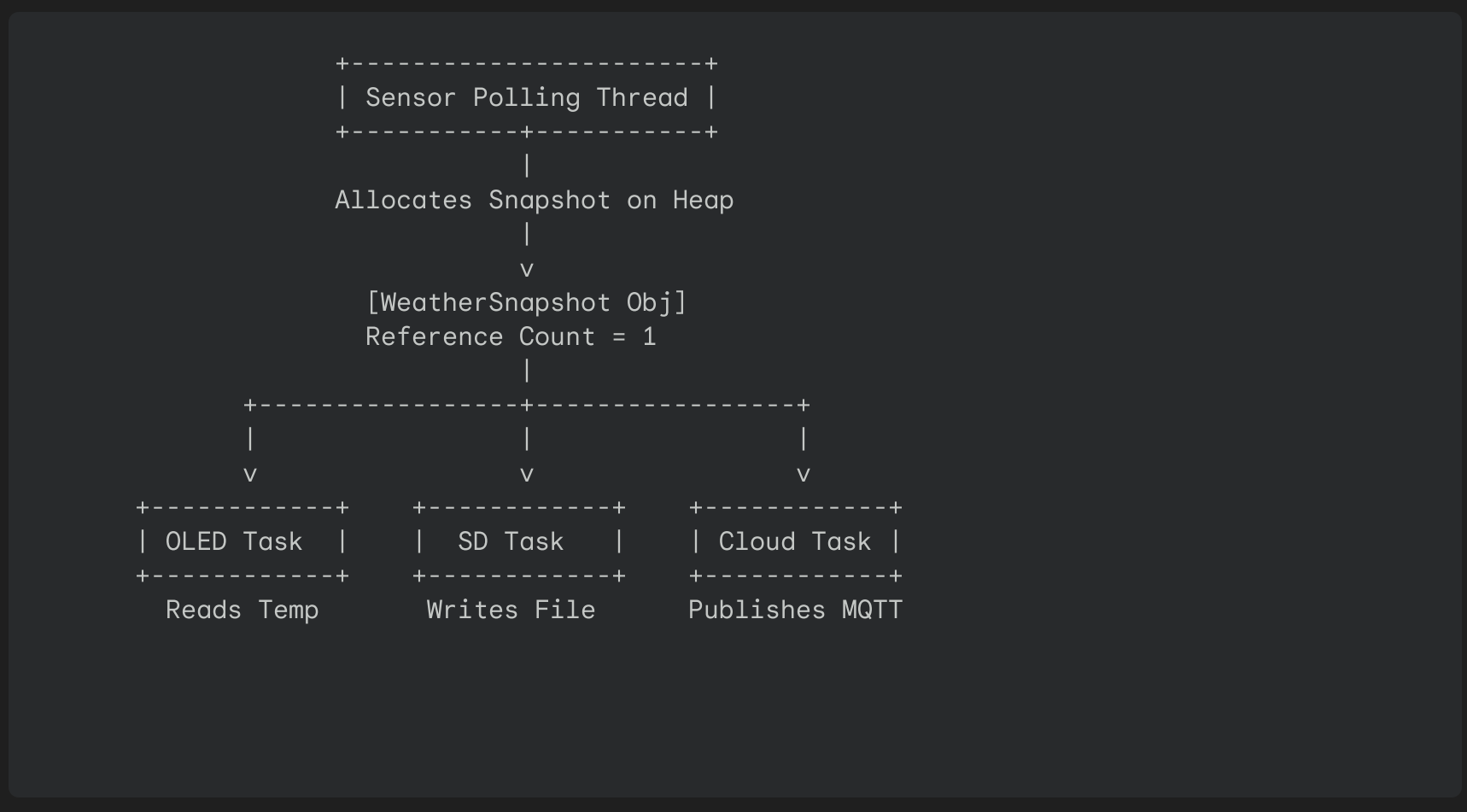
Imagine an RTOS-based IoT weather station. A high-priority background thread polls physical hardware sensors at regular intervals (e.g., $1\ \text{Hz}$). When a read completes, it packages the metrics into a single read-only struct containing temperature, humidity, and barometric pressure.

```
struct WeatherSnapshot {
    float temperature; // in °C
    float humidity;    // in %
    float pressure;    // in hPa
    uint64_t timestamp_ms;
};
```

This snapshot must be processed by three independent, concurrent system tasks:

1. **OLED Display Task:** Renders current metrics on an SPI OLED screen.
2. **SD Card Logging Task:** Writes snapshots to an internal SPI Flash/SD Card file.

3. **AWS IoT Cloud Publisher Task:** Serializes snapshots via JSON over MQTT/WiFi.



The Architectural Problem

The tasks run at completely different speeds. The OLED task updates instantly, the SD write can block for a few milliseconds, and the network broadcast might experience major TCP latency.

- If you use a **raw pointer**, which task is responsible for executing `delete pointer;` when it finishes? If Task A deletes it while Task C is still trying to write to the WiFi socket, Task C will read freed memory and the CPU will suffer a hard fault or Segmentation Fault.
- If you pass by **value** (copying), you are duplicating memory across three queues, which is incredibly wasteful on low-power microcontrollers.

The Modern C++ Solution

Use `std::shared_ptr<const WeatherSnapshot>`. This guarantees zero-copy, read-only safety. Each task receives its own copy of the shared pointer, incrementing the reference count. When each task finishes its work, its local pointer goes out of scope, decrementing the count. The memory is freed the exact microsecond the slowest thread completes.

Code Implementation

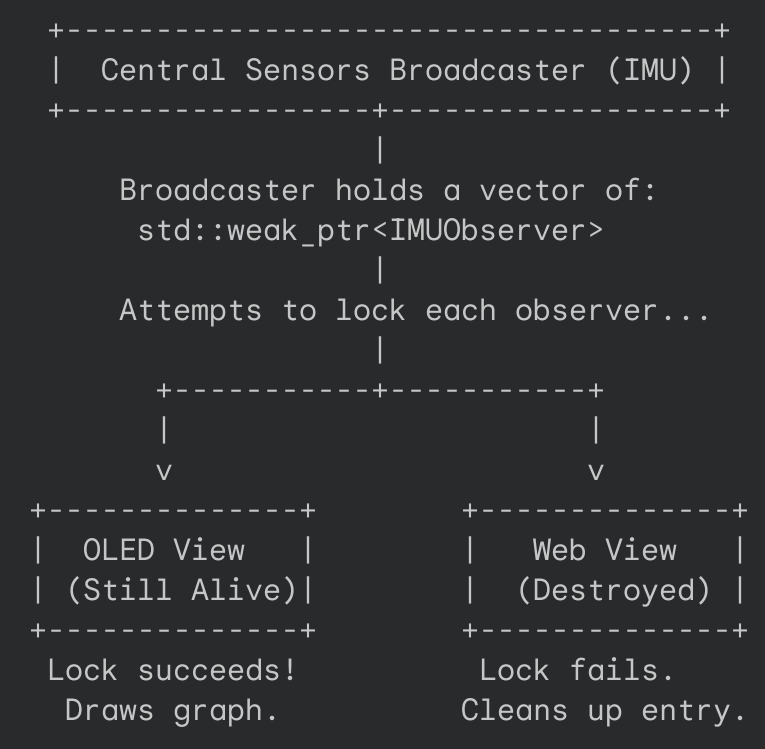
```
#include <iostream>
#include <memory>
#include <thread>
#include <chrono>

struct WeatherSnapshot {
    float temperature;
    float humidity;
    float pressure;
    uint64_t timestamp_ms;

    WeatherSnapshot(float t, float h, float p, uint64_t ts)
        : temperature(t), humidity(h), pressure(p), timestamp_ms(ts) {
        std::cout << "[ALLOC] Snapshot created at " << timestamp_ms << " ms\n";
    }

    ~WeatherSnapshot() {
        std::cout << "[FREE] Snapshot deleted cleanly from memory.\n";
    }
};

// Simulated tasks running on different threads/cores
void oledDisplayTask(std::shared_ptr<const WeatherSnapshot> snapshot) {
    // Local copy of pointer increments the count
    std::this_thread::sleep_for(std::chrono::milliseconds(5)); // Super fast
    std::cout << "  -> [OLED Task] Updated UI. Temp: " << snapshot->temperature << " C\n";
}
```

```
void sdLoggerTask(std::shared_ptr<const WeatherSnapshot> snapshot) {
    std::this_thread::sleep_for(std::chrono::milliseconds(30)); // Medium speed
    std::cout << "  -> [SD Task] Snapshot written to flash log.\n";
}

void wifiPublisherTask(std::shared_ptr<const WeatherSnapshot> snapshot) {
    std::this_thread::sleep_for(std::chrono::milliseconds(80)); // Slow network delay
    std::cout << "  -> [WiFi Task] Sent telemetry payload to AWS IoT Cloud.\n";
}

int main() {
    std::cout << "=== SYSTEM BOOT: IoT WEATHER SNAPSHOT RUN ===\n\n";

    {
        // 1. Thread spawns a new sensor snapshot
        std::shared_ptr<const WeatherSnapshot> latestReading =
            std::make_shared<const WeatherSnapshot>(24.85f, 62.4f, 1012.3f, 45000ULL);

        std::cout << "Active Owners: " << latestReading.use_count() << "\n\n";

        // 2. Dispatch tasks (In an RTOS, this copies pointers into OS queues)
        // Here we simulate concurrent execution by passing the shared_ptr to threads
        std::thread t1(oledDisplayTask, latestReading);
        std::thread t2(sdLoggerTask, latestReading);
        std::thread t3(wifiPublisherTask, latestReading);

        // Main loop can immediately drop its local ownership to continue polling sensors
        latestReading.reset();
        std::cout << "[Main Thread] Dropped local owner pointer. System waiting for tasks...\

        t1.join();
        t2.join();
        t3.join();
    }

    std::cout << "\n=== DIAGNOSTIC RUN ENDED ===" << std::endl;
    return 0;
}
```

3. `std::weak_ptr` Use Case: Asynchronous Observer Pattern for Debug Widgets

Consider an ESP32 drone or automated robotics hub. It has a central **Sensors Broadcaster Service** that reads the onboard IMU (gyroscope and accelerometer) and broadcasts alerts if an anomalous motion spike or collision risk is detected.

The system allows debugging tools (like an OLED graph widget, a terminal CLI monitor, or a web status dashboard) to temporarily register as "Observers" so they can draw data in real-time.



The Architectural Problem

Observers can be created and destroyed dynamically at runtime. For example, a technician might plug in a serial debug monitor, look at a diagnostic screen for 10 seconds, and close it.

- If the `SensorsBroadcaster` held a standard list of `std::shared_ptr<IMUObserver>`, the observers could **never be deleted**. The central service would keep a persistent reference, preventing their destructors from ever running (a memory leak!).
- If the `SensorsBroadcaster` held raw pointers, and a technician closed a debug window, the broadcaster would eventually try to send data to a deallocated memory address, triggering a system crash.

The Modern C++ Solution

The central service maintains a registry of `std::weak_ptr<IMUObserver>`.

1. When the service wants to broadcast, it calls `.lock()` on the observer.
2. If the debug screen still exists, `.lock()` temporarily promotes the pointer to a valid `std::shared_ptr` so the broadcaster can safely transmit metrics.
3. If the user closed the debug screen, `.lock()` returns `nullptr`. The broadcaster safely realizes the view is dead and removes it from its list.

Code Implementation

```
#include <iostream>
#include <memory>
#include <vector>
#include <algorithm>

// The Interface that debugging widgets must implement
class IMUObserver {
public:
    virtual void onTelemetryUpdate(float pitch, float roll) = 0;
    virtual ~IMUObserver() = default;
};

// A concrete Observer (OLED Display view)
class OledGraphicView : public IMUObserver {
public:
    void onTelemetryUpdate(float pitch, float roll) override {
        std::cout << "  [OLED View] Drawing graph -> Pitch: " << pitch << ", Roll: " << roll
    }
    ~OledGraphicView() { std::cout << "  [OLED View] Destructor: Widget closed & cleaned up.`
};

// The central Broadcaster service
class SensorBroadcaster {
private:
    std::vector<std::weak_ptr<IMUObserver>> m_Observers;

public:
    void registerObserver(std::weak_ptr<IMUObserver> observer) {
        m_Observers.push_back(observer);
        std::cout << "[Broadcaster] New observer registered.\n";
    }

    void broadcast(float pitch, float roll) {
        std::cout << "\n[Broadcaster] Broadcasting telemetry payload to registered observers

        // Iterate and filter out dead observers
        auto it = m_Observers.begin();
        while (it != m_Observers.end()) {
            // Attempt to promote the weak pointer
            if (auto sharedObserver = it->lock()) {
                sharedObserver->onTelemetryUpdate(pitch, roll);
                ++it;
            } else {
                // The observer was destroyed in background! Clean up vector.
                std::cout << "[Broadcaster] Dead observer detected. Purging from registry...`
                it = m_Observers.erase(it);
```

```
            }
        }
    }
};

int main() {
    std::cout << "=== SYSTEM BOOT: DRONE TELEMETRY BROADCAST ===\n";

    SensorBroadcaster imuService;

    // Create our graphical screen widget on the stack
    auto oledScreen = std::make_shared<OledGraphicView>();

    // Register it as a weak observer so we don't hold it hostage in memory
    imuService.registerObserver(oledScreen);

    // Initial sensor sweep
    imuService.broadcast(1.2f, -0.4f);

    std::cout << "\n[Simulator] User closes the OLED screen widget...\n";
    // Deallocate the observer manually by releasing our owner pointer
    oledScreen.reset();

    // Second sensor sweep (Broadcaster detects the observer has departed!)
    imuService.broadcast(3.4f, -1.8f);

    std::cout << "\n=== SYSTEM SHUTDOWN ===" << std::endl;
    return 0;
}
```

Summary Reference Table

| **Pointer** | **Who "Owns" the Resource?** | **Embedded Use Case Example** | **Hardware Safety Guarantee** | **Cost / Overhead** | | `std::unique_ptr` | Exactly one single owner module. | A physical UART port, SPI HAL driver, DMA memory channel, RX packet buffer. | Prevents duplicate driver initialization, multi-thread race conditions, and illegal concurrent access. | **Absolutely Zero.** Compiles down to a raw pointer. | | `std::shared_ptr` | Multiple tasks sharing lifetime control. | A global DMA log buffer, a static sensor Snapshot read by logger + network. | Prevents silent memory leaks and "use-after-free" faults between concurrent RTOS Tasks. | Small reference counting tracking block allocated on the heap. | | `std::weak_ptr` | Nobody. A safe "spectator." | Cache buffers, observer callbacks, live sensor registers, parent-child nodes. | Prevents memory leaks by breaking dynamic circular locks / observer traps. | Extremely tiny (compares against the control block). |